

0 The One Problem Transformers Solve

Computers understand numbers, not words. Every ML approach to text asks the same question:

Core Question

How do I turn text into numbers in a way that preserves enough meaning to tell categories apart?

TF-IDF answers: count the important words. Transformers answer: understand what the words mean in context.

```
TF-IDF: "mortgage lending"      -> sparse 50,000-dim vector (word counts)
        "real estate financing" -> completely different sparse vector
        Cosine similarity = 0.0   (model sees NO relationship)

Transformer: "mortgage lending"  -> [0.21, -0.43, 0.87, ...] 768-dim
            "real estate financing" -> [0.23, -0.41, 0.85, ...] 768-dim
            Cosine similarity = 0.94 (model sees near-identical meaning)
```

1 Step 1 — Tokenization

Text is split into tokens — subword units, not just full words. Each token maps to an ID, then to an initial embedding vector.

```
"provides renewable energy generation through solar installations"

Tokens: ['provides', 'renewable', 'energy', 'generation',
        'through', 'solar', 'installations']

IDs:    [4821, 9043, 1822, 3301, 1205, 6741, 2984]

Each ID -> 768-dim vector (initial embedding, before any context)
```

Key Point

The token identity never changes across all 12 layers. Only the vector representing it evolves.

2 Step 2 — Attention (The Core Mechanism)

Attention answers one question for every token:

Attention Definition

How significant is every other token to understanding this one?

This is computed using three learned projections per token:

Q (Query)	What am I looking for? 'asks'
K (Key)	What do I advertise about myself? 'presents' — the label on the filing cabinet
V (Value)	What information do I actually contribute once attended to? 'contributes' — the documents inside the cabinet

K vs V — The Critical Distinction

K and V both belong to each token but serve completely different roles. K determines HOW MUCH attention flows. V determines WHAT INFORMATION travels when it does.

```
Token: "solar" in "provides renewable energy generation through solar installations"
```

```
Keys (what each token advertises):
```

```
"renewable" K -> "I am a sustainability modifier"
```

```
"energy" K -> "I am a power/utilities domain signal"
```

```
"solar" K -> "I am a specific technology type" <- highest relevance
```

```
Query "generation" computes dot products with all Keys:
```

```
Q_generation . K_solar = 0.82 (highest)
```

```
Q_generation . K_energy = 0.71
```

```
Q_generation . K_renewable = 0.68
```

```
Q_generation . K_through = 0.05 (structural word, low)
```

```
After softmax -> attention weights: [solar=0.41, energy=0.22, renewable=0.21, ...]
```

```
Values (what each token contributes when attended to):
```

```
V_solar -> photovoltaic tech, renewable energy sector, clean energy domain
```

```
V_energy -> power sector, utilities classification
```

```
V_renewable -> sustainability, clean energy domain
```

```
Output for "generation":
```

```
0.41 * V_solar + 0.22 * V_energy + 0.21 * V_renewable + ...
```

```
-> now knows it is SOLAR ENERGY generation, not nuclear, not coal
```

Softmax — Turning Scores Into Probabilities

Raw attention scores are any real numbers. Softmax converts them into a probability distribution summing to 1.0, while amplifying differences so attention is focused rather than diffuse.

```
Raw scores: [2.1, 1.4, 0.8, -0.3]
Exponentiate: [8.17, 4.06, 2.23, 0.74] sum = 15.20
Softmax: [0.537, 0.267, 0.147, 0.049] sum = 1.000
```

```
Property 1: preserves ordering (higher raw -> higher probability)
Property 2: amplifies differences (top gets 0.537, bottom gets 0.049 = 11x gap)
Property 3: handles negatives cleanly (e^-0.3 = 0.74, still positive)
Property 4: differentiable (gradients can flow through it during training)
```

Attention Formula

```
Attention(Q, K, V) = softmax( Q * K^T / sqrt(d_k) ) * V
```

```
Q * K^T      -> (seq_len x seq_len) matrix: every token vs every token
/ sqrt(d_k)  -> scale to prevent huge values in high dimensions
softmax(...) -> convert to probability distribution (rows sum to 1)
* V          -> weighted sum of Value vectors
```

Result: each token now carries a context-enriched representation

3 Step 3 — Multi-Head Attention

One attention head can only focus on one type of relationship at a time. Multi-head attention runs H independent heads in parallel, each specialising in a different relationship type.

```
d_model = 768 (full embedding dimension)
H        = 12 (number of heads)
d_k      = 64 (dimension per head = 768 / 12)
```

Each head gets its own projection matrices:

```
Q_h = x * W_Qh (768 x 64) <- different W per head
K_h = x * W_Kh (768 x 64)
V_h = x * W_Vh (768 x 64)
```

```
Each head runs full attention in its own 64-dim subspace
12 outputs (each seq_len x 64) -> concatenate -> (seq_len x 768)
Final linear projection W_O (768 x 768) -> integrates all heads
```

What Each Head Learns (Emergent Specialisation)

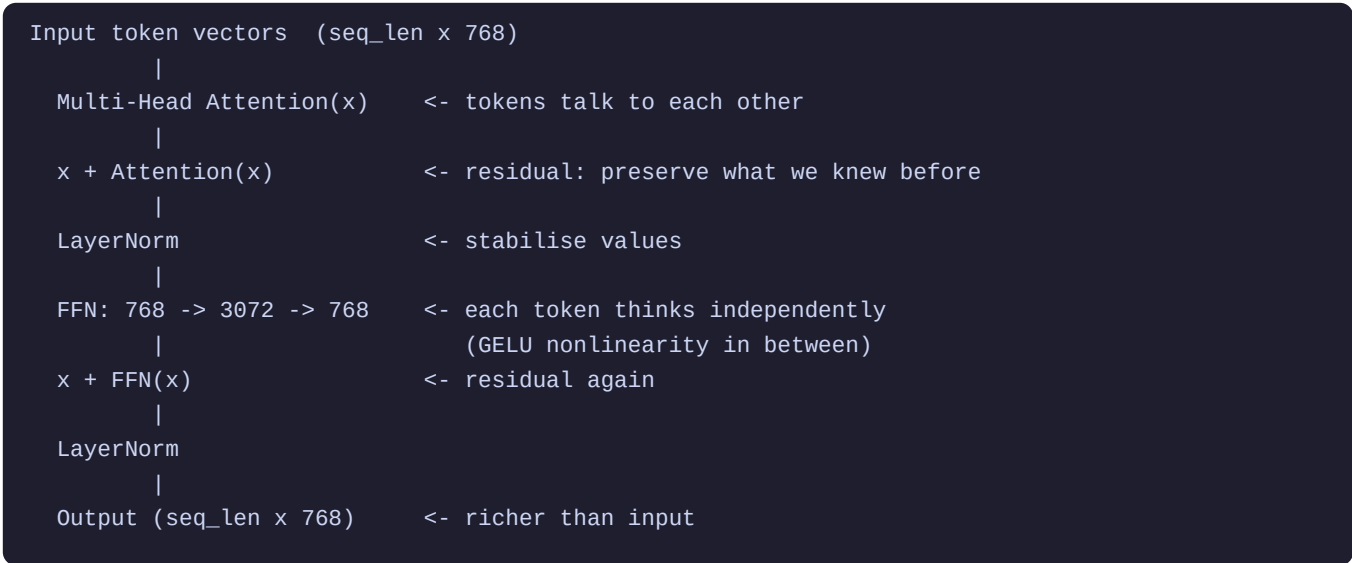
Head type	What it tracks
-----------	----------------

Syntactic	verb-object, subject-verb relationships
Semantic	word meaning similarity, synonyms
Domain	industry signals, topic clustering
Positional	adjacent token relationships
Coreference	same entity referred to differently
Negation	"not profitable" vs "profitable"

Nobody programmed these specialisations. They emerged from backpropagation optimising prediction error across billions of examples.

4 One Full Transformer Layer

Each of the 12 layers has this structure:



Attention	Horizontal — tokens communicate across the sequence
FFN	Vertical — each token processed in depth independently Stores factual/domain knowledge recalled from pretraining
Residual (+x)	Ensures earlier information is never lost Keeps gradients healthy during backpropagation
LayerNorm	Keeps vector values in stable range for next layer

5 Why 12 Layers? (Hierarchical Abstraction)

One layer captures immediate relationships. It takes multiple passes for meaning to propagate and abstract.

Layers 1-3: surface – syntax, word order, basic grammar
 "renewable" modifies "energy"

Layers 4-8: semantic – phrase meaning, entity types
 "leveraged buyout" = private equity activity

Layers 9-12: abstract – task-relevant classification signals
 "this entity belongs in Private Equity sector"

Your classifier sits on layer 12 output.
 It receives the most abstract, task-relevant representation.

Why Not 6 or 24?

BERT-small	4 layers	14M params	weaker generalisation
BERT-base	12 layers	110M params	sweet spot for most tasks
BERT-large	24 layers	340M params	stronger, diminishing returns
GPT-3	96 layers	175B params	very strong, massive cost

6 How The Model Learns — Backpropagation

All those weight matrices (W_Q , W_K , W_V , W_{FFN}) start random. Backpropagation is how they become meaningful.

PRETRAINING TASK (Masked Language Modelling):
 Input: "The company provides [MASK] lending and home loan services"
 Target: "mortgage"

FORWARD PASS:
 tokens -> 12 layers -> classifier -> softmax -> probability over vocabulary
 Predicted: "commercial" (0.31) Correct: "mortgage" (0.04) <- wrong

LOSS:
 $\text{cross_entropy} = -\log(0.04) = 3.22$ <- high, model was very wrong

BACKWARD PASS (chain rule):
 Loss -> layer12 weights -> layer11 weights -> ... -> layer1 weights
 Each weight gets a gradient: which direction reduces loss?

WEIGHT UPDATE:
 $\text{new_weight} = \text{old_weight} - \text{learning_rate} * \text{gradient}$
 Repeat billions of times -> weights converge to encode language meaning

Emergent Property

Nobody told the model what 'meaning' is. It discovered semantic structure as a side effect of predicting missing words efficiently across billions of sentences.

7 From Token Vectors To Sentence Embeddings

After 12 layers every token has a rich 768-dim vector. To classify a whole company description you need one vector. This is called pooling.

Strategy 1 – CLS token:

BERT prepends [CLS] token whose final vector is trained to summarise the whole sentence

Strategy 2 – Mean pooling (used by sentence-transformers):

Average all token vectors at final layer
Often better for semantic similarity tasks

Result: one 768-dim vector per company description

similar meaning -> nearby vectors -> small cosine distance

8 Your Classification Pipeline (Phase 6)

Company description (text)

|
Sentence transformer encoder (all-mpnet-base-v2 or finbert)
-> 12 attention layers + FFN
-> mean pooling

|
768-dim dense embedding vector

|
Classifier head (logistic regression or small MLP)
-> trained on your 53,000 labeled examples

|
Industry prediction (1 of 145 classes)

Encoder: FROZEN (you inherit all pretraining knowledge for free)

Classifier: TRAINED (learns where 145 clusters sit in embedding space)

Why Embeddings Close The F1 Gap From 0.598 → ~0.72+

Adjacent industries	Better representations separate similar industries
Synonym blindness	"mortgage" and "financing" are now close in space
Tail classes	Pretrained knowledge fills sparse training data gap
Singleton classes	Few-shot LLM layer on top (21 classes)

9 The Full Picture In One View

TEXT

```
|
v Tokenise -> token IDs -> initial embedding vectors
|
v Layer 1: Multi-Head Attention (12 heads x 64-dim subspaces)
|         -> tokens communicate, context gathered
|         + residual + LayerNorm
|         FFN: 768->3072->768 (GELU) -> deep per-token processing
|         + residual + LayerNorm
|
v Layer 2 ... Layer 12: same structure, ever-richer representations
|
v Pool (mean) -> one 768-dim sentence embedding
|
v Classifier head -> softmax -> probability over N classes
|
```

PREDICTION

Training: backprop flows right-to-left through this entire graph,
nudging every weight to reduce cross-entropy loss.
Repeat billions of times -> meaning emerges.

10 Concept Cheat Sheet

Concept	One-line definition
Token	Atomic unit of text. Identity fixed, vector evolves across layers.
Embedding	Dense vector where similar meaning = nearby coordinates.
Q (Query)	What this token is looking for from others.
K (Key)	What this token advertises — determines attention routing.
V (Value)	What this token actually contributes when attended to.
Attention	Significance of every token w.r.t. every other token.
Softmax	Converts raw scores to probabilities summing to 1. Amplifies differences.
Multi-head	H parallel attention heads, each specialising in a different relationship type.
FFN	Per-token deep processing. Stores factual/domain knowledge.
Residual connection	$x + \text{layer}(x)$. Preserves earlier info. Keeps gradients healthy.
Layer	One round of attention + FFN. 12 layers = 12 levels of abstraction.

Backpropagation	Chain rule backwards. Assigns credit to each weight. Drives learning.
Pretraining	Train on billions of text examples. Meaning emerges as side effect.
Fine-tuning	Continue training on your specific task. Adapts representations.
Feature extraction	Freeze pretrained encoder. Only train classifier head. Fast and cheap.
Emergent property	Structure that arises from optimisation pressure, not explicit programming.

11 Why This Matters For AI Roles

What You Now Understand

- Why TF-IDF hits a ceiling (no semantics)
- How attention routes information between tokens
- Why K and V serve different roles
- How 12 layers build hierarchical abstraction
- Why meaning is an emergent property
- How backprop shapes all weights from prediction error
- When to use frozen encoder vs fine-tuning

Where This Shows Up In Roles

- Evaluating embedding model choices for your domain
- Diagnosing why a model fails on tail classes
- Designing RAG pipelines (retrieval = cosine similarity in embedding space)
- Explaining transformer decisions to non-technical stakeholders
- Choosing fine-tune vs prompt vs RAG for a given problem
- Debugging attention-related failure modes